

Gold Dice RL

Trabajo Práctico 1 – Reinforcement Learning

Agustín Chmielewski Juan Pablo Saldungaray Uriel Slavkin

Inteligencia Artificial y Neurociencias – UTDT

Cómo encaramos el problema

Al leer el enunciado nos quedaron tres preguntas: cuánto se puede obtener si se juega bien, qué parte de la observación necesita recordar el agente y cómo representar **SCORE**, que permite elegir cualquier cantidad de oro, dentro de una tabla con pocas acciones.

Uno de nosotros viene de competir en ICPC y reconoció un problema de horizonte finito. Antes de entrenar planteamos una programación dinámica (DP). Esta decisión cambió el trabajo: además de entrenar un agente, pudimos calcular una referencia y medir cuánto costaba cada simplificación.

Por qué este juego admite una DP

El ambiente cumple cuatro condiciones que permiten resolverlo hacia atrás:

1. la partida dura 30 turnos;
2. conocemos las probabilidades de dados y tormentas;
3. conocemos el efecto y el costo de cada acción;
4. el futuro depende de un estado pequeño.

Definimos

$$U_t(g, n, b, e) = \text{puntos esperados antes de tirar en el turno } t,$$

donde g es oro, n dados, b bonus y e escudos. Los puntos acumulados no entran en el estado: suman al resultado, pero no modifican tiradas, precios ni tormentas. El valor guardado se incorpora al calcular la transición siguiente.

Después de la tirada observamos su suma R y su máximo M . Para cada acción legal a calculamos de forma determinista la recompensa inmediata r_a y el estado posterior F_a . Luego integramos la tormenta y consultamos el turno siguiente:

$$U_t(s) = \mathbb{E}_{R,M} \left[\max_{a \in A(s,R,M)} (r_a + \mathbb{E}_{\text{tormenta}} U_{t+1}(F_a)) \right].$$

El caso base es el turno 30. No habrá otra tirada, por lo que conviene convertir todo el oro en puntos. Recorremos $t = 30, 29, \dots, 1$ y guardamos sólo dos capas. Calculamos la distribución conjunta (R, M) por convolución, sin muestrear partidas. Para **SCORE(k)** usamos un máximo prefijo que evalúa todas las cantidades en $O(G)$, en lugar de $O(G^2)$.

El archivo `sol_dp.cpp` implementa esta recurrencia en 92 líneas de C++17. Con los límites usados en el análisis imprime **642,452 puntos** desde el estado inicial. La DP no juega el torneo ni aprende de la experiencia; funciona como referencia calculada con conocimiento completo del ambiente.

Una primera intuición que descartamos

Pensábamos que reducir los cientos de valores posibles de **SCORE(k)** destruiría información. Resolvimos de nuevo la DP permitiendo sólo **SCORE(todo)** y obtuvimos el mismo valor dentro de la precisión usada. La política calculada cobra al final. El agente aprendido trabaja entonces con seis opciones: pasar, puntuar todo, comprar un dado, mejorar los dados, comprar un escudo o guardar el mejor dado.

El agente de Reinforcement Learning

Estado y recompensa

La observación original tiene nueve campos. Nosotros aprendemos sobre el estado posterior a la acción, o *afterstate*:

(turno, oro, dados, bonus, escudos, guardado).

Una compra produce un efecto determinista; el azar aparece después, con la tormenta y la próxima tirada. Dos pares estado-acción que llegan al mismo *afterstate* comparten experiencia. Sacamos los puntos acumulados porque no afectan el futuro; `roll_sum` ya se sumó al oro y `roll_max` sólo interviene al evaluar `STORE_BEST_DIE`.

El oro no tiene un límite declarado. Usamos una grilla no uniforme: conserva cada entero entre 0 y 96, donde se concentran los precios, y separa más los valores altos. Interpolamos entre nodos vecinos. Esta representación distingue 41 de 42 cuando 42 habilita una compra.

Conservamos la recompensa del ambiente: `SCORE` entrega como recompensa la cantidad convertida en puntos y las demás acciones entregan cero. Como gran parte de la recompensa llega al final, inicializamos el valor con

$$\Phi = \text{oro} + \text{guardado} + 5 \text{escudos} + \text{turnos restantes} \cdot \text{dados} \cdot (3,5 + \text{bonus}).$$

La tabla aprende una corrección sobre Φ . Esta cuenta valora la caja y la producción esperada si no se comprara nada más. No contiene la probabilidad de tormenta ni decide cuándo conviene comprar; el agente aprende esas diferencias al jugar.

Algoritmo e hiperparámetros

El modelo entregado es **Q-Learning tabular sobre afterstates con Double Q-Learning**. Mantiene dos tablas. En cada actualización, una selecciona la mejor continuación y la otra calcula su valor. Elegimos esta variante porque una sola tabla empezó a empeorar al aumentar los episodios, un efecto compatible con el sesgo de maximización sobre estimaciones ruidosas.

Parámetro	Valor	Decisión
Episodios	1.500.000	mejor checkpoint cada 100.000
γ	1	horizonte finito, sin descontar puntos tardíos
ϵ	0,15 $\rightarrow$ 0,02	descenso durante el 60 % inicial
Escala de α	0,03	actualización por peso visitado
α mínimo	0,002	evita congelar valores poco frecuentes
Evaluación	4.000 partidas	semillas separadas del entrenamiento

Durante la evaluación, `GoldDiceAgent` carga `artifacts/gold_dice_agent.pkl`, deja de explorar y elige la acción con mayor recompensa inmediata más valor futuro.

Resultados

Evaluamos 20.000 partidas por agente. La Figura 1 compara las medias; las líneas finas muestran intervalos de confianza del 95 %.

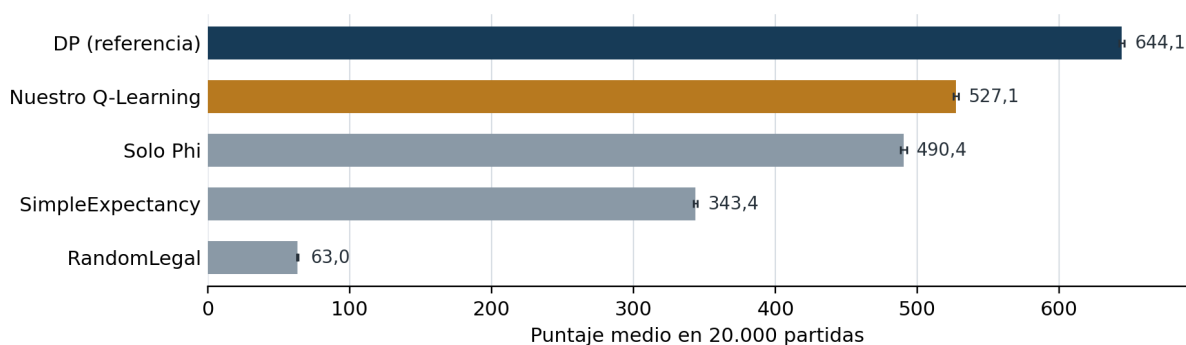


Figura 1: Puntaje medio. La DP aparece como referencia y no como competidor.

Nuestro agente obtuvo **527,06** puntos, frente a 343,40 de `SimpleExpectancy` y 63,03 de `RandomLegal`. Representa el 82 % del valor calculado por la DP. En tres rangos de semillas consiguió 527,06, 526,27 y 525,42; esas diferencias entran en el error estadístico.

La representación aportó más que repetir episodios. Usar Φ sin valores aprendidos produce 490,41 puntos, mientras que el entrenamiento suma cerca de 37. Una tabla Q convencional con *buckets* y topes chicos obtuvo entre 293 y 365. Esos límites agrupaban estados distintos del final de la partida y no podían expresar una política adecuada.

Qué nos permitió saber la DP

La DP respondió preguntas que el puntaje por sí solo no contesta. Sabemos que 527,06 supera los baselines y también sabemos que quedan unos 116 puntos hasta la referencia. Pudimos calcular el costo de cada decisión comparando el valor de la acción elegida con el valor de la mejor acción según la DP. Llamamos **puntos perdidos por una decisión** a esa diferencia; vale cero cuando ambas elecciones coinciden.

La pérdida principal aparece cerca del turno 23. La política de la DP compra un escudo, conserva el oro y cobra en el turno 30. Nuestro agente compra pocos escudos y cobra antes. Probamos trazas de elegibilidad, más exploración y hasta cinco millones de episodios. Ninguna variante cerró la brecha. La secuencia útil exige dos decisiones coordinadas: comprar el escudo y luego ahorrar. La exploración ε -greedy cambia una acción por vez y visita poco esa continuación completa.

Cómo evaluaríamos sin una DP

Este juego ofrece una situación poco común en Reinforcement Learning: conocemos el modelo y el estado es lo bastante chico para enumerarlo. En un problema con imágenes, estados continuos, dinámica desconocida o muchos agentes, la tabla crecería de forma exponencial y tampoco conoceríamos $P(s' | s, a)$. Allí no podríamos calcular U^* .

Sin la DP habríamos conservado tres conclusiones sólidas:

- nuestro agente supera los dos baselines bajo el protocolo de la cátedra;
- mantiene su media en semillas de desarrollo, control y prueba pública;
- las ablaciones muestran qué representación y qué hiperparámetros rinden mejor.

También habríamos perdido información. No podríamos afirmar que el agente alcanza el 82 % del potencial, medir los 116 puntos restantes ni localizar el error del escudo mediante valores óptimos. Trataríamos al mejor resultado observado como una referencia empírica, no como un máximo. Reportaríamos intervalos de confianza, curvas de aprendizaje y comparaciones con el mismo presupuesto de episodios. Las conclusiones serían relativas: “mejor que estos baselines y estas variantes”, sin afirmar cercanía al óptimo.

Esta diferencia separa el rol de la DP del rol del agente. La DP aprovechó que la cátedra nos entregó un modelo pequeño y conocido. Q-Learning resolvió la tarea pedida: aprender una política a partir de interacciones y jugar luego sin consultar esa DP.

Archivos entregados

La cátedra proporcionó `config.py`, `env.py`, `agents.py`, `render.py`, `evaluate_agents.py` y `run_example.py`. Mantuvimos la lógica de `config.py` y `env.py`; agregamos `GoldDiceAgent` dentro de `agents.py`.

Sólo sumamos tres archivos Python propios:

Archivo	Responsabilidad
<code>afterstate.py</code>	acciones legales, transición determinista, grilla de oro y Φ
<code>value_table.py</code>	consulta, interpolación, actualización y persistencia de la tabla
<code>train_qlearning.py</code>	entrenamiento Double Q-Learning y selección del checkpoint

`sol_dp.cpp` contiene la referencia de programación dinámica y el archivo `gold_dice_agent.pkl` contiene los pesos entrenados. Los Markdown `01 REGLAS OCULTAS.md`, `02 RESULTADOS.md` y `03 QUE CONSTRUIAMOS.md` guardan los casos borde, las tablas completas y el inventario de funciones que no entran en estas tres carillas.